

쇼핑몰 앱

1 개요 및 학습목표

프로젝트 소개

1.

개요 및 학습목표



쇼핑몰 앱

- 여러 뷰타입의 Json을 동적으로 List에 표현
- List 공통화
- DI 적용
- Coroutine, Paging3 적용

구현에 필요한 기술들

1.

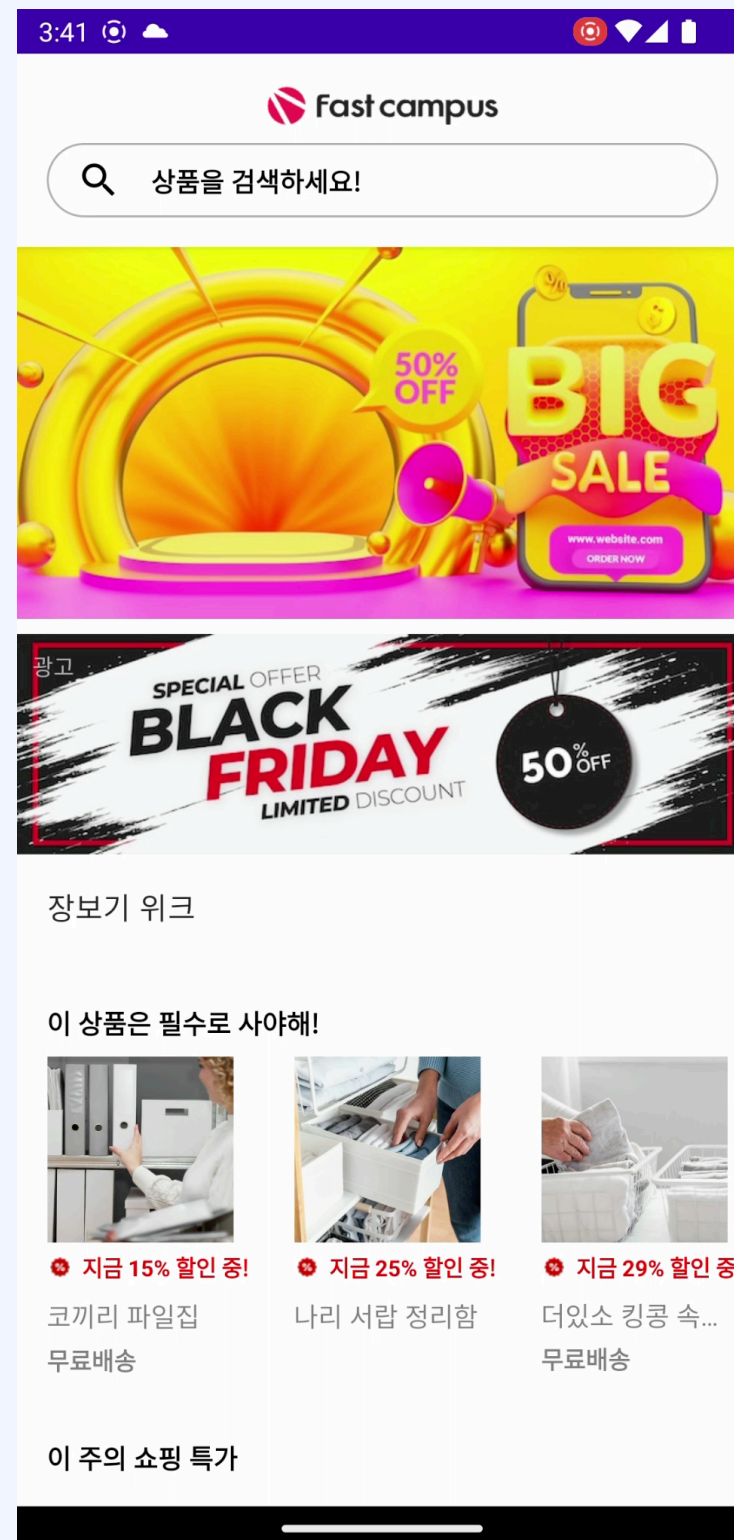
개요 및 학습목표

- MVVM
- Hilt
- Coroutine
- Flow
- Paging3
- JsonSerializer

완성 예제

1.

개요 및 학습목표



- CoordinatorLayout으로 AppBar 접히는 부분 구현
- 복잡한 ViewType의 Json 파싱
- 파싱한 데이터를 동적으로 처리
- API 내용의 변화에 클라이언트 코드 수정 X
- Coroutine, Flow 사용
- DI(Hilt) 적용

쇼핑몰 앱

2 사용 기술

Hilt 란?

2. Hilt

Google의 Dagger를 기반으로 만든 Dependency Injection 라이브러리

Android App에 특화된 DI

- Android class에 의존성 주입을 지원하고 생명 주기를 자동으로 관리

Hilt 란?

2.
사용 기술

Application

```
@HiltAndroidApp  
class ExampleApplication : Application() { ... }
```

Activity

```
@AndroidEntryPoint  
class ExampleActivity : AppCompatActivity() { ... }
```

Inject

```
class AnalyticsAdapter @Inject constructor(  
    private val service: AnalyticsService  
) { ... }
```

Hilt 란?

2.

사용 기술

Android Class에 종속성 주입 범위

- `Application` (`@HiltAndroidApp` 을 사용하여)
- `ViewModel` (`@HiltViewModel` 을 사용하여)
- `Activity`
- `Fragment`
- `View`
- `Service`
- `BroadcastReceiver`

Hilt 란?

Android Class용으로 생성된 Component 생명주기

생성된 구성요소	생성 위치	소멸 위치
SingletonComponent	Application#onCreate()	Application 소멸됨
ActivityRetainedComponent	Activity#onCreate()	Activity#onDestroy()
ViewModelComponent	ViewModel 생성됨	ViewModel 소멸됨
ActivityComponent	Activity#onCreate()	Activity#onDestroy()
FragmentComponent	Fragment#onAttach()	Fragment#onDestroy()
ViewComponent	View#super()	View 소멸됨
ViewWithFragmentComponent	View#super()	View 소멸됨
ServiceComponent	Service#onCreate()	Service#onDestroy()

Hilt 란?

Component Scopes

Android 클래스	생성된 구성요소	범위
Application	SingletonComponent	@Singleton
Activity	ActivityRetainedComponent	@ActivityRetainedScoped
ViewModel	ViewModelComponent	@ViewModelScoped
Activity	ActivityComponent	@ActivityScoped
Fragment	FragmentComponent	@FragmentScoped
View	ViewComponent	@ViewScoped
@WithFragmentBindings 주석이 지정된 View	ViewWithFragmentComponent	@ViewScoped
Service	ServiceComponent	@ServiceScoped

Hilt 란?

그 밖의 Annotation

@Module

Hilt 모듈인지 여부를 판단

@InstallIn

Component 범위를 지정

@Binds

인터페이스 삽입

@Provides

인스턴스 삽입

@Qualifier

동일한 유형에 대해 여러 결합 제공

@ApplicationContext Application의 context를 제공하는 한정자

Coroutine 이란?

2. Coroutine

비동기적으로 실행되는 코드를 간소화하기 위해
Android에서 사용할 수 있는 동시 실행 설계 패턴

Kotlin 1.3에 추가
일종의 경량 스레드
사용 방법

```
dependencies {  
    implementation 'org.jetbrains.kotlinx:kotlinx-coroutines-android:1.3.9'  
}
```

Coroutine 이란?

2. Coroutine

코루틴 구성 요소

CoroutineContext : 코루틴이 실행될 Context

CoroutineScope : 코루틴을 제어할 수 있는 범위

Builder : 코루틴을 실행하는 함수.

Coroutine 이란?

CoroutineContext

코루틴 처리를 어떻게 할 것 인지에 대한 요소들의 집합.

CoroutineContext 요소

Dispatcher : 코루틴을 처리할 스레드를 Setting 하고 할당하는 역할

Main	메인 스레드
IO	I/O 작업하는데 최적화되어 있는 스레드
Default	CPU 사용량이 많은 작업을 할 때 실행
Unconfined	다른 Dispatchers와 달리 특정 스레드를 지정하지 않음

Job : 생성된 코루틴을 컨트롤. (생명 주기, 부모 자식 관계 정리 및 관리)

Coroutine 이란?

2. Coroutine

CoroutineScope

코루틴 처리를 어떻게 할 것 인지에 대한 요소들의 집합.

종류	범위
GlobalScope	Application lifecycle
CoroutineScope	Scope안의 작업이 끝날 때까지
ViewModelScope	ViewModel lifecycle

Coroutine 이란?

2. Coroutine

Builder

코루틴을 생성하는 메소드

launch	Job()을 반환. 결과는 Return 하지 않음(실행 후 망각)
async	Deferred<T> 객체를 리턴. await() 로 결과를 Return
withContext	Context를 변환. await() 없이 결과를 Return
runBlocking	결과를 반환 할 때까지 현재 스레드를 중단

suspend function

suspend 키워드가 붙은 코루틴 안에서 실행시키려는 함수

Flow 란?

2. Flow

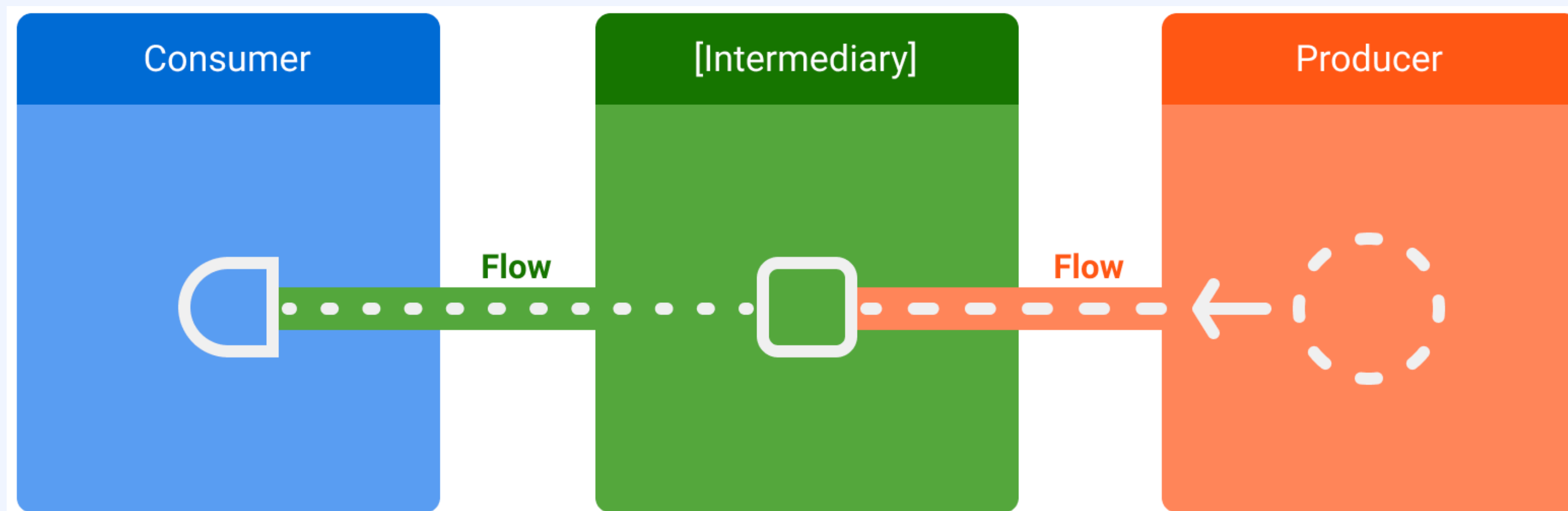
Flow

여러 값을 순차적으로 내보낼 수 있는 유형

생산자 : 스트림에 추가되는 데이터를 생산

중개자 : 스트림에 내보내는 각각의 값이나 스트림 자체를 수정

소비자 : 스트림의 값을 사용



Flow 란?

2. Flow

Flow 생성

```
class NewsRemoteDataSource(  
    private val newsApi: NewsApi,  
    private val refreshIntervalMs: Long = 5000  
) {  
    val latestNews: Flow<List<ArticleHeadline>> = flow {  
        while(true) {  
            val latestNews = newsApi.fetchLatestNews()  
            emit(latestNews) // Emits the result of the request to the flow  
            delay(refreshIntervalMs) // Suspends the coroutine for some time  
        }  
    }  
}  
  
// Interface that provides a way to make network requests with suspend functions  
interface NewsApi {  
    suspend fun fetchLatestNews(): List<ArticleHeadline>  
}
```

Flow 란?

2. Flow

스트림 수정

```
class NewsRepository(  
    private val newsRemoteDataSource: NewsRemoteDataSource,  
    private val userData: UserData  
) {  
    /**  
     * Returns the favorite latest news applying transformations on the flow.  
     * These operations are lazy and don't trigger the flow. They just transform  
     * the current value emitted by the flow at that point in time.  
     */  
    val favoriteLatestNews: Flow<List<ArticleHeadline>> =  
        newsRemoteDataSource.latestNews  
            // Intermediate operation to filter the list of favorite topics  
            .map { news -> news.filter { userData.isFavoriteTopic(it) } }  
            // Intermediate operation to save the latest news in the cache  
            .onEach { news -> saveInCache(news) }  
}
```


Flow 란?

2. Flow

Flow 수집

```
class LatestNewsViewModel(  
    private val newsRepository: NewsRepository  
) : ViewModel() {  
  
    init {  
        viewModelScope.launch {  
            // Trigger the flow and consume its elements using collect  
            newsRepository.favoriteLatestNews.collect { favoriteNews ->  
                // Update View with the latest favorite news  
            }  
        }  
    }  
}
```

Paging3 이란?

2. Paging3

Paging3

데이터를 순차적으로 불러 올 수 있는 Jetpack 라이브러리

라이브러리 아키텍처

